Ans. to Ques. No: 01

(a)

i) Given the array A[] = {6, 4, 8, 2, 3}

To sort an array of size N in ascending order, we have to iterate over the array and compare the current element (key) to it's predecessor, if the key element is smaller than it's predecessor, then we need to compare it to the elements before and then move the greater elements one position up to make space for swapped element.

Now for the given array, A[] = {6, 4, 8, 2, 3}

6	4	8	2	3

First Pass: Initially, we will compare the first two elements of the array in insertion sort. Here 6 is greater than 4 hence they are not in ascending order and 6 is not in its correct position. Thus we will swap 6 and 4. So for now 4 is stored in a sorted subarray.

4	6	8	2	3

Second Pass: Moving on to the next two elements, we have 6 and 8 for the comparison. As 6 is less than 8 so both 6 and 8 in their correct position and no

swapping will occur. 6 will also stored in a sorted sub-array along with 4.

4	6	8	2	3

Third Pass: Now the two elements that are present in the sub array is 4 and 6. moving on, we have 8 and 2 for the comparison. Since 8 is greater than 2, then they are not in their correct position. So we need to swap them.

4	6	2	8	3

After swapping, 6 and 2 are not sorted, Thus we will swap again

4	2	6	8	3

Here again we will swap 4 and 2 as they are not sorted.

2	4	6	8	3

Here, 2 is at it's correct position.

ii) From the question before, we have seen that after third pass in insertion sort, element 2 has reached the first position in the sub array.

2	4	6	8	3

The total movement needed to reach element 2 the first position in the array is 4.

If the cost associated with movement of each element is 10 dollars, the total cost needed for element 2 to reach the first position = (4 x 10) dollars

= 40 dollars.

(b)

i) Given that,

$n >= 3$

```
A()
{
  while (n > 1)
      n = n/3;
}
```

time complexity = $O(\log_3 n)$

ii) B()

```
{ for(i=n/2 ; i<=n; i++)  ---- n/2
    for(j=1; j<=n/2 ; j++)  ---- n/2
      for(k=1; k<=n ; k=k*2) ---- log2^n
        printf("Algorithm");
}
```

time complexity, $O\left(\frac{n}{2} * \frac{n}{2} * \log_2 n\right)$

1) C

Best case is the function which performs the minimum number of steps on input data of n elements. Worst case is the function which performs the maximum number of steps on input data of size n.

The worst case gives us an upper bound on performance. Analyzing an algorithm's worst case guarantees that it will never perform worse than what we determine. Therefore, we know that the other cases must perform at least as well.

2) A

The worst-case input for QuickSort occurs when the chosen pivot element is consistently the smallest or largest element in the array. This results in the algorithm repeatedly partitioning the array into two subarrays of size n-1 and 0, leading to poor performance.

Certainly, let's consider an 8-element array and walk through the worst-case scenario for quicksort. In this scenario, we will consistently choose the smallest or largest element as the pivot, leading to unbalanced partitions.

Array: [8, 7, 6, 5, 4, 3, 2, 1]

If we choose the first element (8) as the pivot, the partitioning steps will be as follows:

1. Pivot: 8. Subarrays: [7, 6, 5, 4, 3, 2, 1] | [8]
2. Pivot: 7. Subarrays: [6, 5, 4, 3, 2, 1] | [7] | [8]
3. Pivot: 6. Subarrays: [5, 4, 3, 2, 1] | [6] | [7] | [8]
4. Pivot: 5. Subarrays: [4, 3, 2, 1] | [5] | [6] | [7] | [8]

At each step, one partition contains only the pivot, and the other partition contains the rest of the elements. This results in a situation where the algorithm doesn't effectively divide the array, leading to a worst-case time complexity of O(n^2) instead of the average case's O(n log n).

In this example, consistently choosing the smallest (or largest) element as the pivot results in unbalanced partitions, making quicksort inefficient for sorting the given array.

Ans to the Q.No- 2(b)

Given that,

$$T(n) = 4T(n/2) + n^3$$

Here,

$$a = 4$$
$$b = 2$$
$$k = 3$$
$$P = 0$$

$$b^k = 2^3 = 8;$$

$$\therefore a < b^k$$

since $P \geq 0$, $T(n) = O(n^k \log^P n)$

$$= O(n^3 \log^0 n)$$

$$= O(n^3).$$

CamScanner

Ans to the Q.NO - 2(c)

Given that,

$f(x) = O(g(x))$ and $g(x) = O[h(x))$ then

$$f(x) = O(h(x))$$

By the definition of Big-O (O), there exists positive constants c, x_0 such that

$$f(x) \leq c \cdot g(x) \text{ for all } x \geq x_0$$

$$\Rightarrow f(x) \leq c_1 \cdot g(x)$$

$$\Rightarrow g(x) \leq c_2 \cdot h(x)$$

$$\Rightarrow f(x) \leq c_1 \cdot c_2 h(x)$$

$$\Rightarrow f(x) \leq c \cdot h(x)$$

Where $c = c_1 \cdot c_2$ By the definition.

$$f(x) = O(h(x))$$

CamScanner

3. a)

Given sequence, P={2,4,2,4,2}, fill up following m and s table,

matrix	A_1	A_2	A_3	A_4
dimension	2×4	4×2	2×4	4×2
	$P_0 \times P_1$	$P_1 \times P_2$	$P_2 \times P_3$	$P_3 \times P_4$

m

	1	2	3	4
1	0	16	32	44
2		0	32	32
3			0	16
4				0

s

	2	3	4
1	1	2	1
2		2	2
3			3

$$m[i,j] = \begin{cases} 0 & i=j \\ \min_{i \le k < j} m[i,k] + m[k+1,j] + P_{i-1} P_k P_j & i<j \end{cases}$$

$m[1,2]$

at $k=1$

$m[1,2] = m[1,1] + m[2,2] + P_0 \cdot P_1 \cdot P_2$

$= 0 + 0 + 2 \times 4 \times 2$

$= 16$

$m[2,3]$

at $k=2$

$m[2,3] = m[2,2] + m[3,3] + P_1 P_2 P_3$

$= 0 + 0 + 4 \times 2 \times 4$

$= 32$

CamScanner

$m[3,4]$ =

at $k=3$

$m[3,4] = m[3,3] + m[4,4] + P_2P_3P_4$

$= 0 + 0 + 2 \times 4 \times 2$

$= 16$

$m[1,3]$

at $k=1$

$m[1,3] = m[1,1] + m[2,3] + P_0P_1P_3$

$= 0 + 32 + 2 \times 4 \times 4$

$= 64$

at $k=2$

$m[1,3] = m[1,2] + m[3,3] + P_0P_1P_4$

$= 16 + 0 + 2 \times 4 \times 2$

$= 32$

$m[2,4]$

at $k=2$

$m[2,4] = m[2,2] + m[3,4] + P_1P_2P_4$

$= 0 + 16 + 4 \times 2 \times 2$

$= 32$

at $k=3$

$m[2,4] = m[2,3] + m[4,4] + P_1P_3P_4$

$= 32 + 0 + 4 \times 4 \times 2$

$= 64$

CamScanner

m[1,4]

at k=1

$m[1,4] = m[1,1] + m[2,4] + P_0 P_1 P_4$

$= 0 + 32 + 2 \times 4 \times 2$

$= 48$

at k=2

$m[1,4] = m[1,2] + m[3,4] + P_0 P_2 P_4$

$= 16 + 16 + 2 \times 4 \times 4$

$= 64$

at k=3

$m[1,4] = m[1,3] + m[4,4] + P_0 P_3 P_4$

$= 32 + 0 + 2 \times 4 \times 2$

$= 48$

CamScanner

3.b).

i) "ATGC" and "TACGTC"

	y_j	A	T	G	C
x_i	0	0	0	0	0
T	0	←0	↖1	←1	←1
(A)	0	↖(1)	←1	←1	←1
C	0	↑(1)	←1	←1	↖2
G	0	↑(1)	←1	↖2	←2
(T)	0	↑1	↖(2)	←(2)	←2
(C)	0	↑1	↑2	←2	↖(3)

longest

Hence, the length of the common subsequence is 3 and the subsequence is ATC

ii) "AT" and "TACGTCGA"

	y_j	A	T
x_i	0	0	0
T	0	↑0	↖1
(A)	0	↖(1)	↑1
C	0	↑(1)	↑1
G	0	↑(1)	↑1
(T)	0	↑1	↖(2)
C	0	↑1	↑(2)
G	0	↑1	↑(2)
A	0	↑1	↑(2)

longest

Hence, the length of the common subsequence is 2 and the subsequence is AT.

CamScanner

iii) "ATG" and "TACGTCGA".

	y_j	A	T	G
x_i	0	0	0	0
T	0	↑0	↖1	←1
(A)	0	↖(1)	↑1	↑1
C	0	↑(1)	↑1	↑1
G	0	↑(1)	↑1	↖2
(T)	0	↑1	↖(2)	↑2
C	0	↑1	↑(2)	↑2
(G)	0	↑1	↑2	↖(3)
A	0	↖1	↑2	↑3

longest

Hence the length of the common subsequence is 3 and the subsequence is AGT

3(c)

Optimal Substructure

If the optimal solution to a problem P, of size n, can be calculated by looking at the optimal solutions to subproblems [p1,p2,...](not all the sub-problems) with size less than n, then this problem P is considered to have an optimal substructure.

Let's understand this by taking some examples. Check whether the below problem follows optimal substructure property or not?

Shortest Path Problem

Problem Statement - Consider an undirected graph with vertices a, b, c, d, e and edges (a, b), (a, e), (b, c), (b, e),(c, d) and (d, a) with some respective weights. Find the shortest path between a and c.

This problem can be broken down into finding the shortest path between a & b and then shortest path between b & c and this can give a valid solution i.e. shortest path between a

We need to break this for all vertices between a & c to check the shortest and also direct edge a-c if exits. So the following problem can be broken down into sub-problems and it can be used to find the optimal solution to the bigger problem(also the subproblems are optimal). So this problem has an optimal substructure.

Longest Path Problem

Problem Statement - For the same undirected graph, we need to find the longest path between a and d.

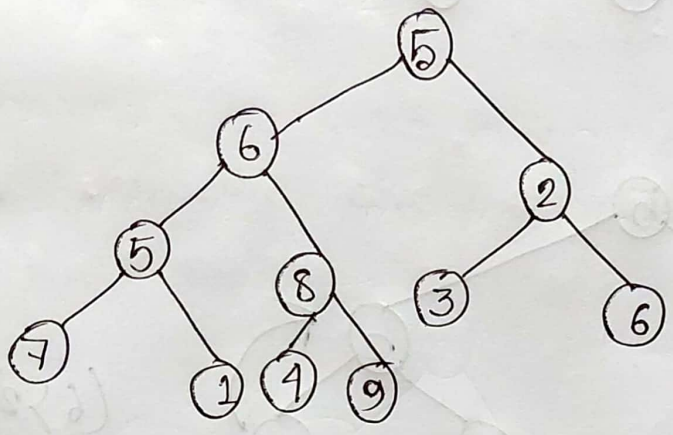
Let us suppose the longest path is a->e->b->c->d, but if we think like the same manner and calculate the longest paths by dividing the whole path into two subproblems i.e. between a & c i.e. (a->e->b->c) and c & d i.e. (c->b->e->a->d), it won't give us a valid(because we need to use non-repeating vertices) longest path between a & d. So this problem does not follow optimal substructure property because the substructures are not leading to some solution.

Answer to the question no-4

4(a)

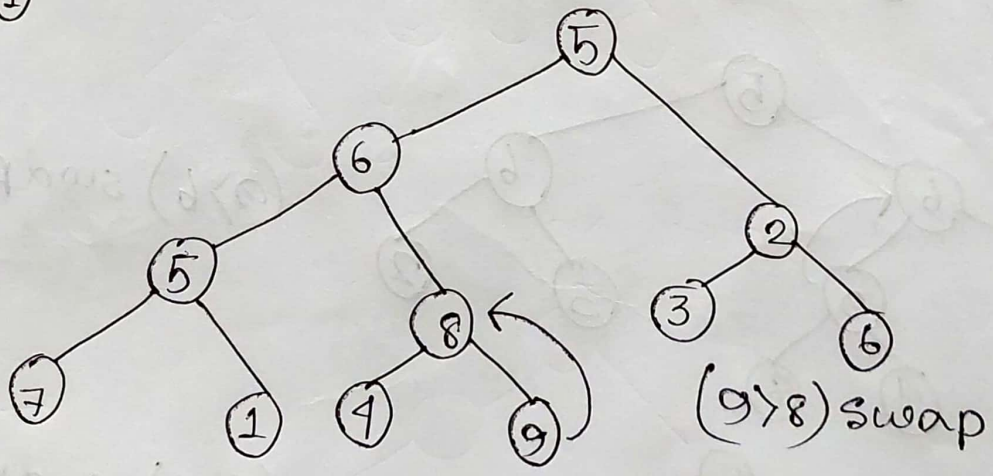
Given the array, $A = \{5, 6, 2, 5, 8, 3, 6, 7, 1, 4, 9\}$

Building a binary tree from the array:

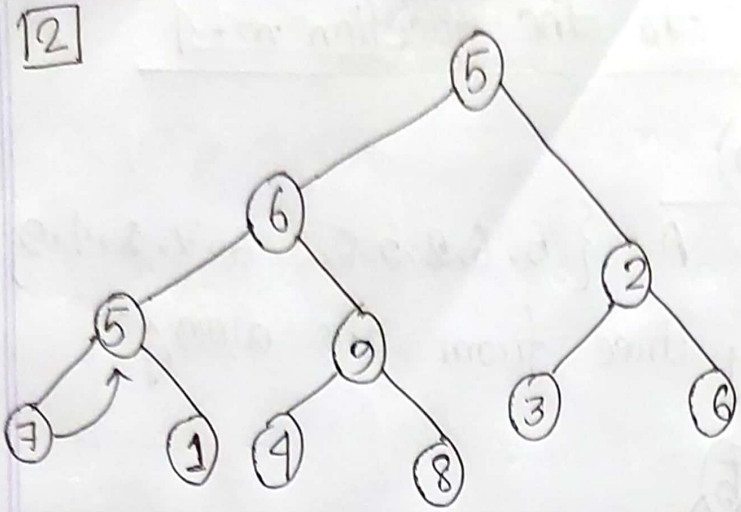


Now building max heap using Build-Max-Heap:

(1)

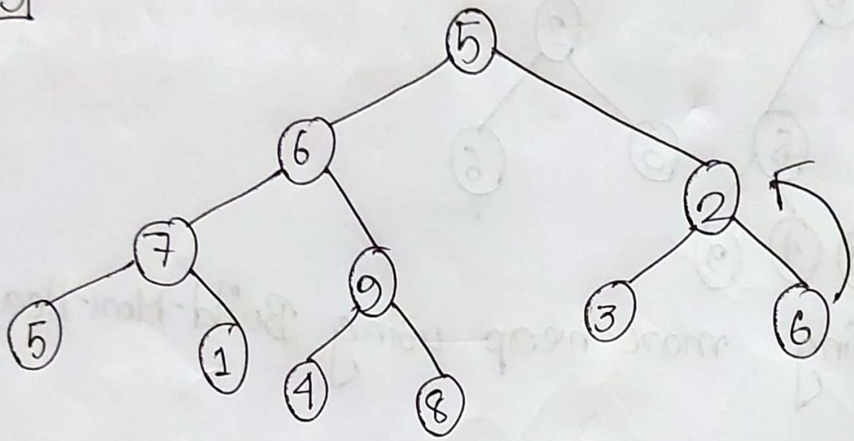


2



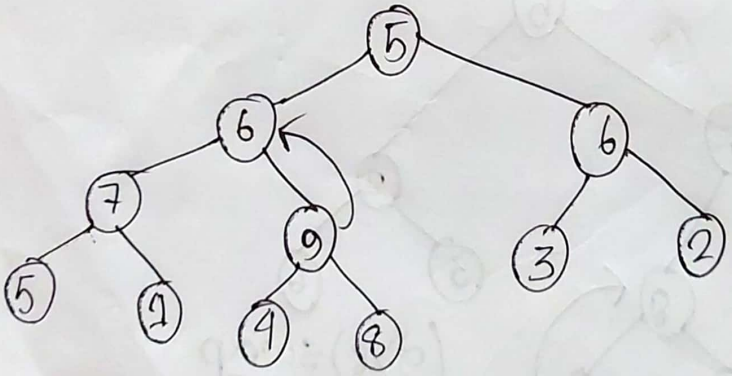
(7>5) swap

3



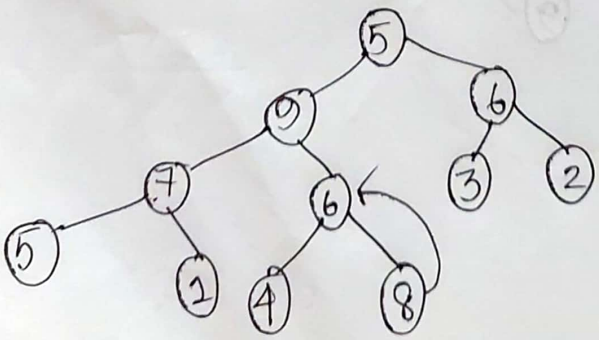
(6>2) swap

4



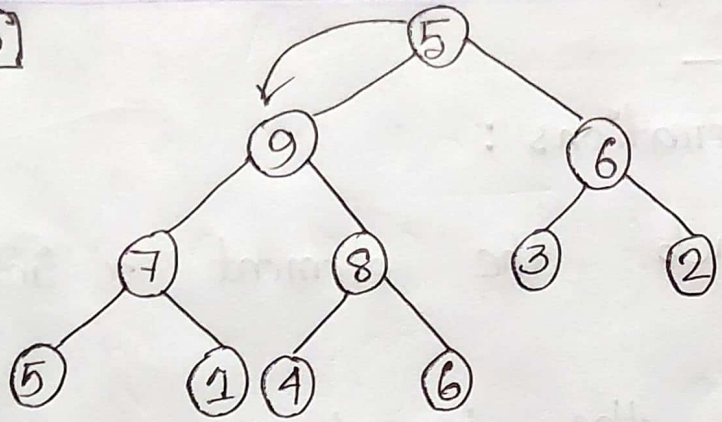
(9>6) swap

5



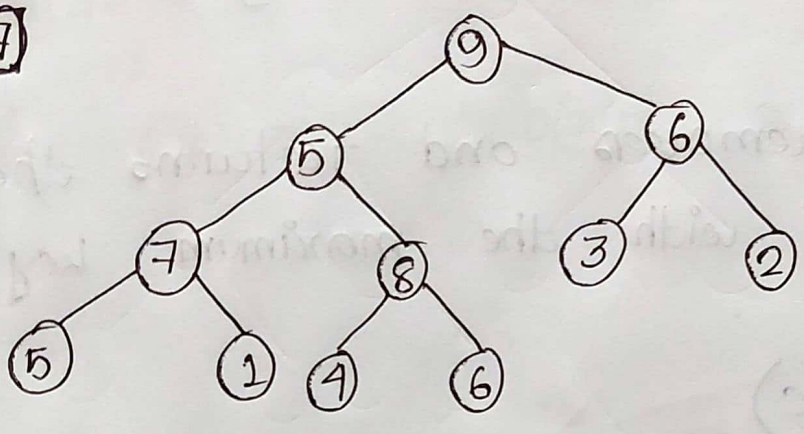
(8>6) swap

[6]

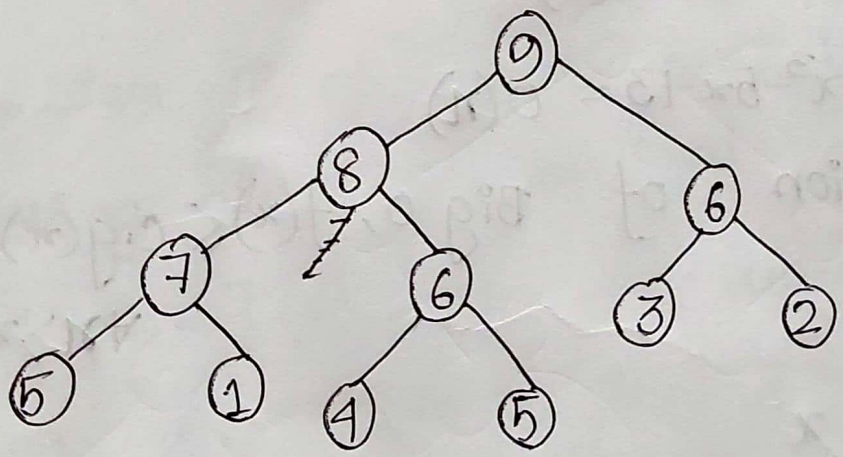


(9 > 5) swap

[7]



The final Max heap tree will be -



4(b)

Priority queue operations :

1. Insert(S, x) inserts the element x into set S.
2. Maximum(S) returns the element of S with the maximum key.
3. ExtractMax(S) removes and returns the element of S with the maximum key.

4(c)

$f(x) = 4x^3 - 5x + 3 \neq O(x^2)$

Let's assume; $4x^3 - 5x + 3 = O(x)$

By the definition of Big O, $f(x) \leq c \cdot g(x)$ $\forall x \geq x_0$

$$4x^3 - 5x + 3 \leq c \cdot x$$

$$\Rightarrow \frac{4x^3 - 5x + 3}{x} \leq c$$

$$\Rightarrow 4x^2 - 5 + 3/x \leq c$$

$$\Rightarrow 4x^2 + 3/x \leq c + 5$$

$\Rightarrow 4x^2 + \frac{3}{x} \leq C \quad (C = C+5) \quad \cdots (i)$

Let $n \to \infty$,

In eqn (i) L.H.S $4x^2 + \frac{3}{x}$ keep increasing but R.H.S is constant.

After, x_0 ; L.H.S $\geq$ R.H.S

So, $4x^3 - 5x + 3 \neq 0 (x^4)$

Q-4(d)

Mention the sequence of four steps to develop a dynamic Programming algorithm.

⇒ 1. Characterize the structure of an optimal solution.

2. Recursively define the value of an optimal solution.

3. Compute the value of an optimal solution, typically in a bottom-up fashion.

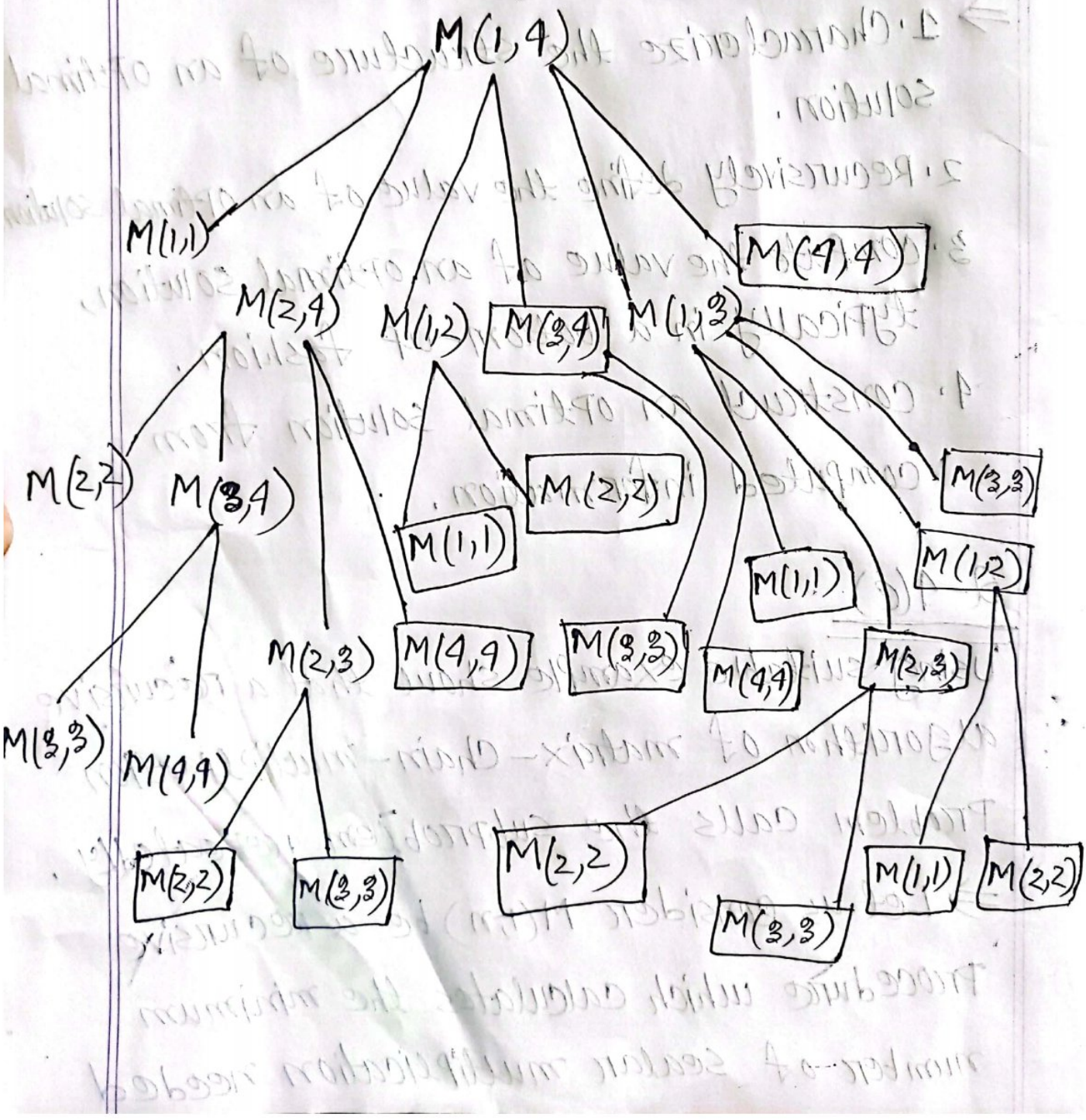
4. Construct an optimal solution from computed information.

Q-4(e)

Using suitable example show that a recursive algorithm of matrix-chain-multiplication Problem calls the subproblem repeatedly.

⇒ Let us consider M(1,n) be a recursive Procedure which calculates the minimum number of scalar multiplication needed

for a matrix chain $\langle A_1, A_2, \ldots A_n \rangle$! The recursive calling tree for M(1,4) be



The redundent callings are boxed, we can see that M(1,2) was called for finding solutions for both M(1,4) and M(1,3).

Hence, we can say that this Problem has overlapping sub-Problem repeatedly.